NYCU-EE IC LAB – SPRING 2026

Lab11 Exercise

Design: mini Image Signal Process(ISP)-APR

Data Preparation

1. Extract test data from TA's directory:
   % tar -xvf ~iclabTA01/Lab11_2026_Spring.tar

Design Description

This project features the hardware implementation of a high-performance, area-efficient 12-bit Mini Image Signal Processor (ISP) pipeline. Designed to process 16 x 16 Bayer RAW input (RGGB pattern), the system performs real-time image reconstruction and enhancement through a series of specialized hardware modules. The pipeline consists of:

- **Black Level Correction (BLC)**
- **Lens Shading Correction (LSC)**
- **Directional 5×5 Defective Pixel Correction (DPC)**
- **3×3 Bilinear Demosaicing**
- **Color Correction Matrix (CCM)**

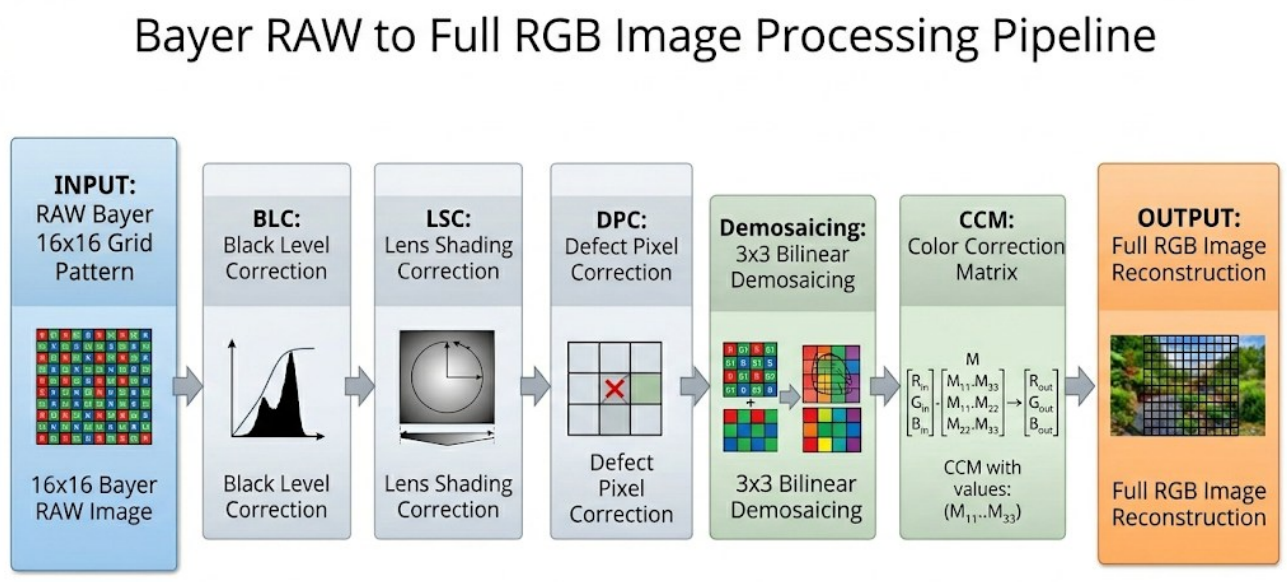


Fig. 1. Mini Image Signal Processor flow

Bayer Raw Image

A **Bayer RAW image** is the direct output from an image sensor before any image processing is applied. In a typical digital camera sensor, each pixel can only measure **one color component** because a **color filter array (CFA) is placed on top of the sensor.** The most commonly used CFA pattern is the **Bayer pattern**, which arranges the color filters in a repeating **RGGB pattern**.

As a result, each pixel location stores **only one color value (R, G, or B)** instead of full RGB information.

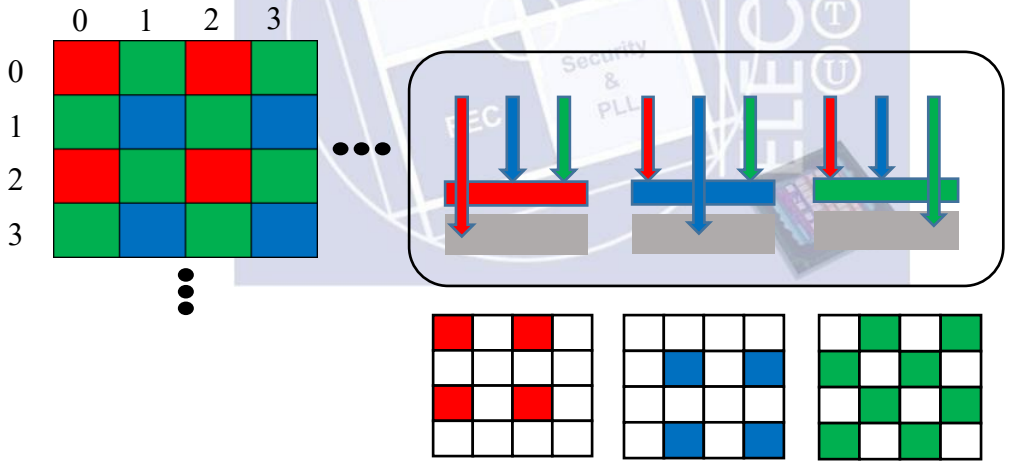


Fig. 2.Bayer Raw Image(RGGB)

The order of the input pixel(Row-major order):

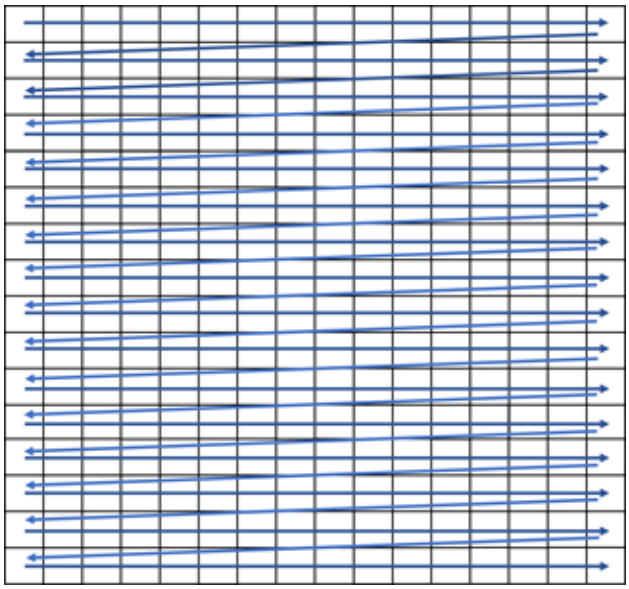
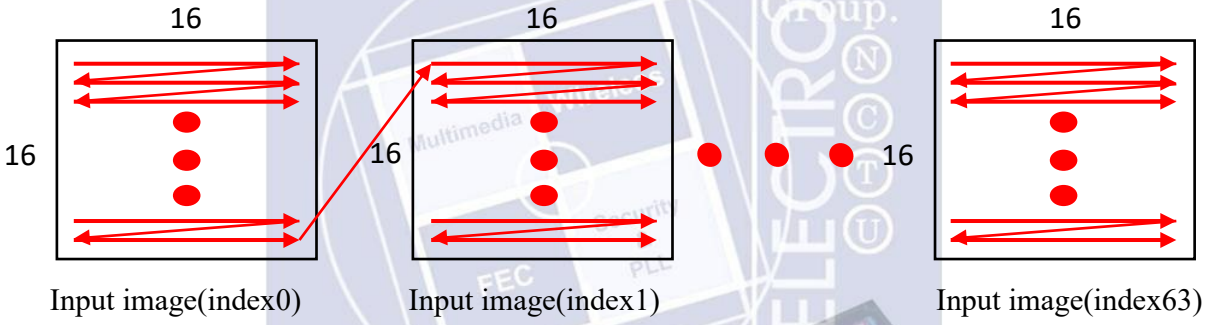


Fig. 3.The order of input pixel

Input Image (NEW)

Each image has a fixed size of 16×16 pixels. Each pixel is stored as an 12-bit unsigned value. The 16×16 frame is provided in raster-scan order from image 0 to image 63.



- SUMA180_4096X12: It is a single-port SRAM that can store 4096 words of 12 bits each. In this lab, you will use four SRAMs of this type to store images: MEM0 stores image 0~15, MEM1 stores image 16~31, MEM2 stores image 32~47 and MEM3 stores image 48~63, respectively.

address	Data
0	image_0[0][0]
1	image_0[0][1]
⋮	⋮
255	image_0[15][15]
256	image_1[0][0]
257	image_1[0][1]
⋮	⋮
4094	image_15[15][14]
4095	image_15[15][15]

MEM0 (image 0~15)

Figure 4. Stored data representation of MEM0

After that, you will receive an 6-bit command. The following is rule about the command:

MSB	Format – (6bits)	LSB
SRAM index(2 bits)	Pattern index(4 bits)	

Black Level Correction (BLC)

Sensor outputs include a constant black level offset due to analog circuitry.

The corrected pixel is computed as:

$$I_{\text{blc}} = \max(0, I_{\text{raw}} - B)$$

Where:

- I_{raw} is the 12-bit sensor input
- B is the programmable black level(RGGB)

In this design, each Bayer channel has an independent black level value:

Channel	Black Level
R (even row, even col)	64
Gr (even row, odd col)	48
Gb (odd row, even col)	52
B (odd row, odd col)	72

Table 1. Black Level of different channels

Lens Shading Correction (LSC)

Lens Shading Correction (LSC) is used to compensate for the vignetting effect, a physical phenomenon where the image brightness gradually decreases from the center toward the corners due to lens characteristics and sensor chief ray angle (CRA) mismatch.SC is performed in the RAW (Bayer) domain and must be completed before the Demosaic process to ensure color consistency.

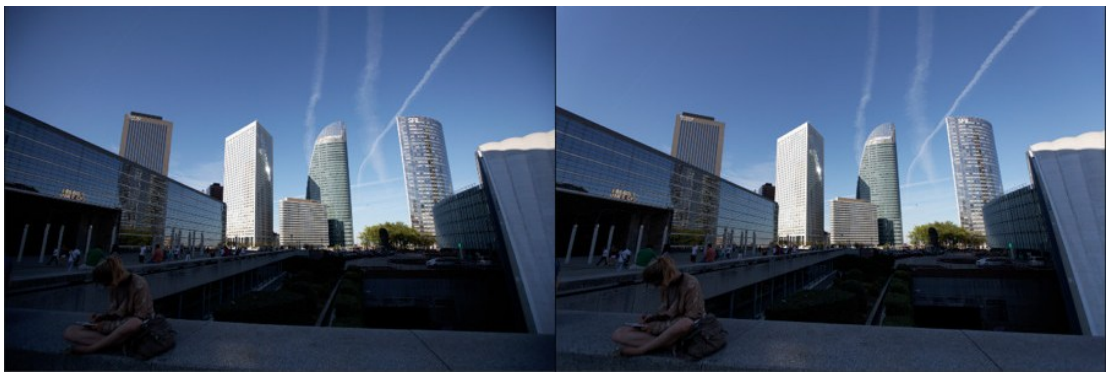
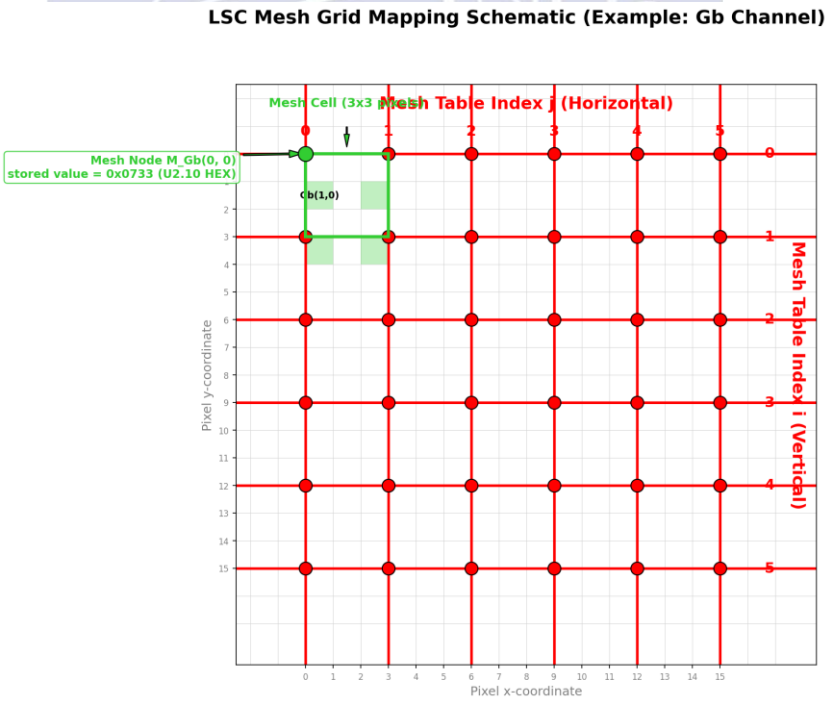


Fig. 5. Comparison of Lens Shading Correction

1. Gain Mesh Specification

To optimize hardware memory, the image is divided into a 5 x 5 grid of cells.

- Mesh Table: At the beginning, param_gain sequentially transmits the mesh gain tables for all four color channels.



- Numerical Format: Mesh gains are stored in Fixed-point U2.10 format.
    - Example: A gain of 1.25x is represented as 1.25 x 1024 = 1280.
    - A gain would be 1.00x to 2.00x, meaning 1024~2048

2. Coordinate Mapping

For a pixel at coordinate (x, y), where x, y ∈ [0,15], the system identifies the surrounding mesh nodes and the relative position within the cell:

- Mesh Node Indices (x_0, y_0):

$$x_0 = min(\lfloor\frac{x}{3}\rfloor,4), y_0 = min(\lfloor\frac{y}{3}\rfloor,4)$$

The four active nodes are: $M(y_0, x_0), M(y_0, x_0 + 1), M(y_0 + 1, x_0), M(y_0 + 1, x_0 + 1)$.

$$\begin{aligned} g_{00} &= M_c(y_0, x_0) \\ g_{01} &= M_c(y_0, x_0 + 1) \\ g_{10} &= M_c(y_0 + 1, x_0) \\ g_{11} &= M_c(y_0 + 1, x_0 + 1) \end{aligned}$$

- Cell Fractional Position (r_x, r_y):

$$r_x = min(x - 3x_0, 2), r_y = min(y - 3y_0, 2)$$

3. Bilinear Interpolation Weights

The interpolation uses 8-bit fractional precision to compute the horizontal and vertical influence:

- Horizontal Weights:

$$d_x = \left\lfloor\frac{r_x \cdot 256 + 1}{3}\right\rfloor$$

$$i_x = 256 - d_x$$

- Vertical Weights:

$$d_y = \left\lfloor\frac{r_y \cdot 256 + 1}{3}\right\rfloor$$

$$i_y = 256 - d_y$$

4. Interpolation and Application

The gain G(x, y) is calculated by blending the four neighbor nodes $(g_{00}, g_{01}, g_{10}, g_{11})$:

$$G(x,y) = \frac{g_{00} \cdot i_x \cdot i_y + g_{10} \cdot i_x \cdot d_y + g_{01} \cdot d_x \cdot i_y + g_{11} \cdot d_x \cdot d_y + 2^{15}}{2^{16}}$$

The final corrected pixel P'(x, y) is calculated by applying the gain to the original RAW pixel P(x, y), followed by a 12-bit saturation clip:

$$P'(x,y) = min(\lfloor\frac{P(x,y) \cdot G(x,y) + 2^9}{2^{10}}\rfloor,4095)$$

Directional 5×5 Defective Pixel Correction (DPC)

Image sensors may contain defective pixels, which produce abnormal values that are much brighter (hot pixels) or much darker (dead pixels) than their surrounding pixels. These defects can be caused by manufacturing imperfections, aging of the sensor, or noise during image capture. If these defective pixels are not corrected, they will appear as visible artifacts such as bright or dark spots in the image, which significantly degrade image quality. Therefore, Defective Pixel Correction (DPC) is applied in the early stage of the ISP pipeline to detect and repair these abnormal pixels.

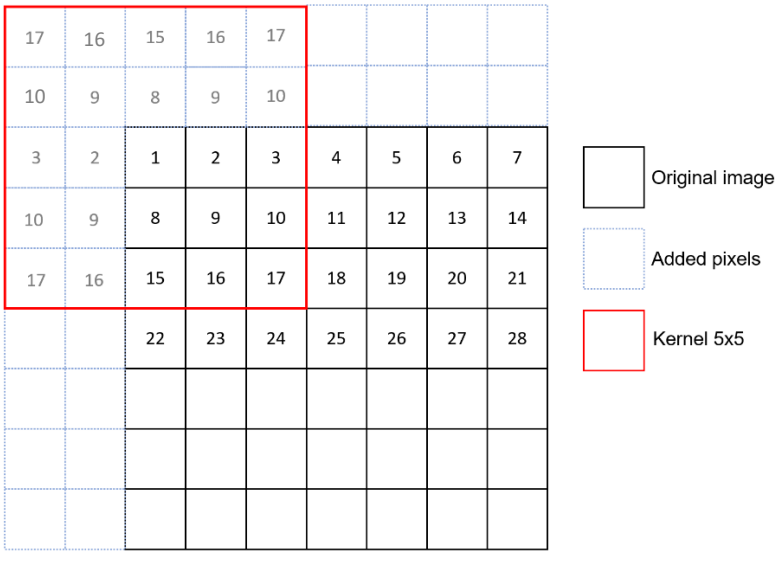


Fig. 6. Comparison of Defective Pixel Correction [1]

1. A 5×5 window is extracted.

   Supplemental: To handle boundary conditions and ensure the output image size remains W×H, reflect padding of 2 pixels is applied to the top, bottom, left, and right edges of the raw image. Let I(x,y) be the pixel at coordinate (x,y), where x in [0, W-1] and y in [0, H-1]. The padded pixel I'(x,y) is defined as:

$$I'(x,y) = \begin{cases} I(-x,y), & \text{if } x < 0 \text{ (Left boundary)} \\ I(2W-2-x,y), & \text{if } x \geq W \text{ (Right boundary)} \\ I(x,-y), & \text{if } y < 0 \text{ (Top boundary)} \\ I(x,2H-2-y), & \text{if } y \geq H \text{ (Bottom boundary)} \end{cases}$$



2. Four directional vectors are formed:

   Four directional vectors are formed from the 5×5 window:

   - **Horizontal (H):** $[H_0, H_1, H_2, H_3]$
   - **Vertical (V):** $[V_0, V_1, V_2, V_3]$
   - **Diagonal 1 (D1):** $[D_{10}, D_{11}, D_{12}, D_{13}]$
   - **Diagonal 2 (D2):** $[D_{20}, D_{21}, D_{22}, D_{23}]$

D_{10}		V_0		D_{23}
	D_{11}	V_1	D_{22}	
H_0	H_1		H_2	H_3
	D_{21}	V_2	D_{12}	
D_{20}		V_3		D_{13}

3. For each of the four orientations (Horizontal, Vertical, and two Diagonals), a **Smoothness Score** is dynamically computed to evaluate the local texture consistency. This is achieved by calculating the **Sum of Absolute Differences (SAD)** between the four neighboring pixels in that direction and the **corresponding median** of the four neighboring pixels:

$$Score_{dir} = \sum_{i=0}^{3} |Pixel_{dir,i} - Median_{dir}|$$

4. The direction with the minimum SAD is selected, and its **median** is designated as the Target. The center pixel **P** is replaced by **Target** if:

$$|P - \text{target}| > \text{TH}$$

Where:

- **P** is the center pixel
- **Target** is median of the direction with the minimum SAD
- **TH** is 320

Tie-Breaking Rule for Equal SAD Scores

If two or more directions produce the same minimum SAD score, the direction is selected according to the following fixed priority order:

Horizontal (H) > Vertical (V) > Diagonal 1 (D1) > Diagonal 2 (D2)

For example, if H and V both have the minimum SAD, H is chosen.

3×3 Bilinear Demosaicing

Each pixel location contains only one color component (R, G, or B).The missing two color components must be reconstructed using neighboring pixels.

- **Interpolation Rules**

1. Boundary Handling & Window Extraction

Let the center pixel be located at coordinate (x, y). To compute the missing color components, we define its 8-neighborhood within a 3×3 window as follows:

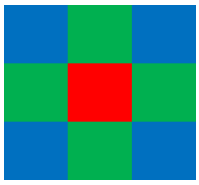
$$\begin{matrix} NW & N & NE \\ W & C & E \\ SW & S & SE \end{matrix}$$

To maintain the original image resolution of W×H, reflect padding of 1 pixel is applied to all four edges. For each pixel P(x,y), a 3×3 window is extracted. If a neighbor's coordinate (x', y') falls outside the image boundaries, the pixel value is mirrored as follows:

- If $x' < 0$, $x' = |x'|$
- If $x' \geq W$, $x' = 2W - 2 - x'$
- (The same logic applies to the y coordinate)

(A) At R Location

At an **R pixel** position, the Red component is original, while the **Green** and **Blue** components are missing and must be interpolated from the 3×3 neighborhood.



1. Green Interpolation

The missing Green value is calculated by averaging the four direct neighbors (North, South, East, West):

$$G = \frac{N + S + E + W}{4}$$

2. Blue Interpolation

The missing Blue value is calculated by averaging the four diagonal neighbors (North-West, North-East, South-West, South-East):

$$B = \frac{NW + NE + SW + SE}{4}$$

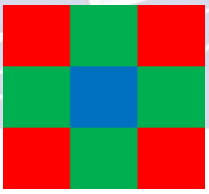
3. Final RGB Output

The final triplet for the pixel located at the Red sensor site is:

$$\begin{cases} R_{\text{out}} = R_{\text{center}} \\ G_{\text{out}} = \dfrac{N+S+E+W}{4} \\ B_{\text{out}} = \dfrac{NW+NE+SW+SE}{4} \end{cases}$$

(B) At B Location

At a **B pixel** position, the Blue component is original, while the **Red** and **Green** components are missing and must be interpolated from the 3×3 neighborhood.



1. Green Interpolation

The missing Green value is calculated by averaging the four direct neighbors (North, South, East, West):

$$G = \frac{N+S+E+W}{4}$$

2. Red Interpolation

The missing Red value is calculated by averaging the four diagonal neighbors (North-West, North-East, South-West, South-East):

$$R = \frac{NW+NE+SW+SE}{4}$$

3. Final RGB Output

The final triplet for the pixel located at the Blue sensor site is:

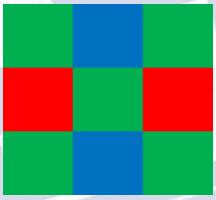
$$\begin{cases} R_{\text{out}} = \dfrac{NW+NE+SW+SE}{4} \\ G_{\text{out}} = \dfrac{N+S+E+W}{4} \\ B_{\text{out}} = B_{\text{center}} \end{cases}$$

(C) At G Location

The interpolation at a **Green pixel** depends on its position within the Bayer grid. There are two distinct cases based on whether the Green pixel is located on a **Red row** or a **Blue row**.

1. G on R Row (G in R-G-R)

At this location, the Green value is original. The horizontal neighbors are **Red**, and the vertical neighbors are **Blue**.



- **Red Interpolation (Horizontal):**

$$R = \frac{W + E}{2}$$

- **Blue Interpolation (Vertical):**

$$B = \frac{N + S}{2}$$

Final Output:

$$\begin{cases} R_{\text{out}} = \dfrac{W + E}{2} \\ G_{\text{out}} = G_{\text{center}} \\ B_{\text{out}} = \dfrac{N + S}{2} \end{cases}$$

---

2. G on B Row (G in B-G-B)

At this location, the Green value is original. The horizontal neighbors are **Blue**, and the vertical neighbors are **Red**.



- **Red Interpolation (Vertical):**

$$R = \frac{N + S}{2}$$

- **Blue Interpolation (Horizontal):**

$$B = \frac{W + E}{2}$$

Final Output:

$$\begin{cases} R_{\text{out}} = \dfrac{N+S}{2} \\ G_{\text{out}} = G_{\text{center}} \\ B_{\text{out}} = \dfrac{W+E}{2} \end{cases}$$

Color Correction Matrix (CCM)

Even after Demosaicing provides a full RGB image, the colors are not yet accurate to human perception. This is due to several physical factors:

- **Spectral Cross-talk:** The Red, Green, and Blue micro-filters on a CMOS sensor are not "perfect." For example, a Red filter might still allow some Green light to pass through. This results in "muddy" or desaturated colors.
- **Sensor Response vs. Human Eye:** The sensitivity of the silicon sensor (CFA) differs significantly from the Human Visual System (HVS).
- **Color Space Alignment:** The RAW RGB values are in the "Sensor RGB" space. CCM is required to transform these values into a standard color space (such as sRGB) to ensure that the red looks "red" and the skin tones look natural.

Mathematically, this is achieved by a 3x3 linear transformation where each output color is a weighted sum of all three input colors, allowing the ISP to "subtract" the overlapping spectral noise.

1. Matrix Linear Combination

The Color Correction Matrix (CCM) converts the input RGB values into corrected RGB values using a 3×3 matrix multiplication.

For each pixel, the input 12-bit RGB values are combined linearly:

$$\begin{bmatrix} R' \\ G' \\ B' \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} & C_{13} \\ C_{21} & C_{22} & C_{23} \\ C_{31} & C_{32} & C_{33} \end{bmatrix} \cdot \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

The floating-point equivalent CCM used in this design is approximately:

$$\mathbf{C} \approx \begin{bmatrix} 1.074 & -0.049 & -0.049 \\ -0.049 & 1.074 & -0.049 \\ -0.049 & -0.049 & 1.074 \end{bmatrix}$$

This matrix slightly increases the target channel while subtracting small contributions from the other channels to improve color accuracy.

2. Fixed-Point Hardware Implementation

In hardware, floating-point numbers are inefficient. Therefore, we use fixed-point integer coefficients. All coefficients are scaled by 2^{10} :

$$\mathbf{C}_{int} = \begin{bmatrix} 1100 & -50 & -50 \\ -50 & 1100 & -50 \\ -50 & -50 & 1100 \end{bmatrix}$$

Before bit-shifting, we add 2^9. Mathematically, this is equivalent to adding 0.5 in the decimal domain to achieve correct rounding to the nearest integer:

$$R_{round} = (C_{int} \cdot R_{raw} + 512)$$

The result is right-shifted by 10 bits to return the values to the original 12-bit dynamic range

$$R_{shift} = R_{rounded} >> 10$$

To prevent data overflow or underflow, the values are constrained to the legal 12-bit range (0 to 4095):

$$R' = max(0, min(R_{shifted}, 4095)$$

I/O specification

Signals name	Direction	Bit Width	Definition
clk	input	1	Clock.
rst_n	input	1	Asynchronous active-low reset.
in_data_valid	input	1	High when input signal is valid.
In_data	input	12	12-bit RAW Bayer pixel input data (RGGB format). One pixel per cycle.
cmd_valid	input	1	High when cmd signal is valid.
cmd	input	6	1
out_valid	output	1	High when output is valid.
r_out	output	8	The 12-bit red channel output will be truncated to the upper 8 bits.
g_out	output	8	The 12-bit green channel output will be truncated to the upper 8 bits.
b_out	output	8	The 12-bit blue channel output will be truncated to the upper 8 bits.

✧ **We removed the original param_gain and fixed it to use the table provided in the attached file lsc_gain_map.txt.**

Specifications

1. Top module name：ISP (Filename: ISP.v)

Reset

2. It is an asynchronous reset and active-low architecture. If you use synchronous reset (reset after clock starting) in your design, you may fail to reset signals.
3. The reset duration must not exceed three clock cycles.
4. The reset check will only be applied to the outputs. You only need to ensure that the outputs are correctly reset. However, if you remove the reset of any internal DFF and it causes the design to fail during the demo, it will be your responsibility.

Input and Output signal

5. The next cmd inputs will come in 1~4 cycles after your out_valid pull down.
6. The latency of your design in each pattern should be less than 1500 cycles. The latency is the clock cycles between the falling edge of the **cmd_valid** and the falling edge of the **out_valid**.
7. For every cycle, the output must simultaneously produce all three color values (R, G, and B).
8. The input and output data are arranged in row-major order.
9. When out_valid is low, the r_out, b_out, and g_out should be zero.

Synthesis

10. The maximum clock period is set to 20 ns, and you can determine the clock period by yourself.
11. The adjustment interval of the cycle time must be 0.1. For example: 19.1, 15.2 are valid values, while 19.12 is not allowed.
12. The synthesis result of data type cannot include any LATCH.
13. Total Cell Area should not larger than 4,000,000 μm2 and the macro area should not less than 1,400,000 μm2.
14. The slack in the timing report should be non-negative and the result should be MET.

Gate level

15. The gate level simulation cannot include any timing violation.

APR

16. You should refer to the APR_flow_2026S.pdf for the complete flow.
17. Floorplanning

- Core size: Defined by user
- Core to IO boundary: Each side must be more than 100
- Hard Macro placement: All hard macro should be in CORE

18. Powerplanning

- Core Ring

(i) Top & Bottom: metal layer must be odd (1,3,…) and width is 9
(ii) Left & Right: metal layer must be even (2,4,…) and width is 9
(iii) Each side must be wire group, interleaving, and at least 5 pairs.

- Stripes

(i) Horizontal: metal layer must be odd (1,3,…) and width is at least 2
(ii) Vertical: metal layer must be even (2,4,…) and width is at least 2
(iii) The maximum distance between two stripes or stripe and edge should be less than 200

- Power pads

(i) Core power pad: at least 4 pairs and well connected to power ring
(ii) IO power pad: at least 4 pairs.

19. Timing analysis results

- Timing slack: No negative slacks after post-route setup/hold time analysis (including SI)

20. Design verification results

- Design Rule Violation (DRV): The DRV of (fanout, cap, tran) should be all 0 after post

21. Route setup/hold time analysis (including SI)
- Layout v.s. Schematic (LVS): NO LVS violations after “verify Connectivity”
- Design Rule Check (DRC): NO DRC violations after “verify DRC”
- IO Filler and core fillers should be added.

22. CHIP area should not larger than 10,000,000 μm2.

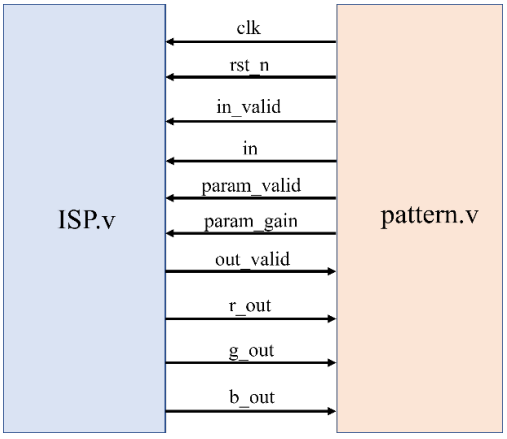
Post-layout simulation

23. The post-simulation cannot include any timing violations without the notimingcheck command.
24. Error message about SRAM is not allowed during post-layout simulation.
25. The latency of post-layout simulation should be the same as that of gate-level simulation, or you will fail in this lab.

else

26. Any words with “error”, “FAIL”, “latch” or “Congratulation” can’t be used as variable name.
27. Ten test cases will be provided, but hidden test cases will be used during the demo.
28. In this lab, you are NOT allowed to use Designware IP.
29. The pattern files will be provided to you directly, so you don't need to generate them yourselves. The pattern only sends the 16384 in_data once at the beginning. After that, the current image to process is selected using cmd. A total of 64 cmd packets will be sent to complete all patterns in one round. The 16384 in_data will not be retransmitted again.

Block Diagram



Note

1. **Grading policy:**
   RTL/Gate-level/Post-level simulation, Synthesis and APR correctness: 70%
   Performance (Chip Area*Cycle Time*Execution Cycle): 30%

2. **Please submit your design through Lab11_2026_Spring /09_SUBMIT/**
   - 1st_demo deadline: 2026/05/25(Mon.) 12:00:00
   - 2nd_demo deadline: 2026/05/27(Wed.) 12:00:00 (score will be 30% off)
   - If uploaded files violate the naming rule, you will get 5 deduct points.

3. Please upload the following files under 09_SUBMIT
   ➢04_MEM_iclabXXX (Folder)
   All memory-related files must be placed directly under this folder. Do not create any subdirectories (e.g., 04_MEM_iclabXXX/04_MEM) to store the files, or your lab will be considered failed. The submitted .lef file should be modified version.
   Memory.v / Memory_WC.db / Memory_BC.lib / Memory_WC.lib / Memory.lef

   ➢ RTL design: ISP_iclabXXX.v
   ➢ filelist.f: filelist_iclabXXX.f
   - The memory path defined in filelist.f should be related path instead of absolute path.
   - No comment contexts are allowed in filelist.f
   - TA strongly recommends the following style to avoid any errors during demo.

```
1   TESTBED.v
2   ../04_MEM/SUMA180_4096X12.v
```

   ➢ APR file (The files will be fetched from 05_APR):
   CHIP_iclabXXX.sdc / CHIP_cts_iclabXXX.sdc / CHIP_iclabXXX.inn / CHIP_iclabXXX.io / CHIP_iclabXXX.v / CHIP_iclabXXX.sdf / CHIP_iclabXXX.inn.dat

   ➢ Please make sure your file tree structure matches the figure below after you execute the 3 commands: 00_tar, 01_submit and 02_check in 09_SUBMIT sequentially. If your files violate the naming rules or directory structure, your lab will be marked as failed. You can use the command: "tree -L <layer>" to know your file tree structure.

```
18:27 iclabTA07@ee25[~/Lab11_2026spring_test/09_SUBMIT]$ tree -L 2
.
|-- Lab11_iclabTA07
|   |-- 04_MEM_iclabTA07
|   |-- CHIP_cts_iclabTA07.sdc
|   |-- CHIP_iclabTA07.inn
|   |-- CHIP_iclabTA07.inn.dat
|   |-- CHIP_iclabTA07.io
|   |-- CHIP_iclabTA07.sdc
|   |-- CHIP_iclabTA07.sdf
|   |-- CHIP_iclabTA07.v
|   |-- GATE_14_iclabTA07.txt
|   |-- ISP_iclabTA07.v
|   |-- POST_20_iclabTA07.txt
|   `-- filelist_iclabTA07.f
|-- Lab11_iclabTA07.tar.gz
`-- Lab11_iclabTA07_1st_demo.tar.gz
```

```
18:29 iclabTA07@ee25[~/Lab11_2026spring_test/09_SUBMIT/Lab11_iclabTA07]$ tree -L 2
.
|-- 04_MEM_iclabTA07
|   |-- SUMA180_4096X12.lef
|   |-- SUMA180_4096X12.v
|   |-- SUMA180_4096X12_BC.lib
|   |-- SUMA180_4096X12_WC.db
|   `-- SUMA180_4096X12_WC.lib
```

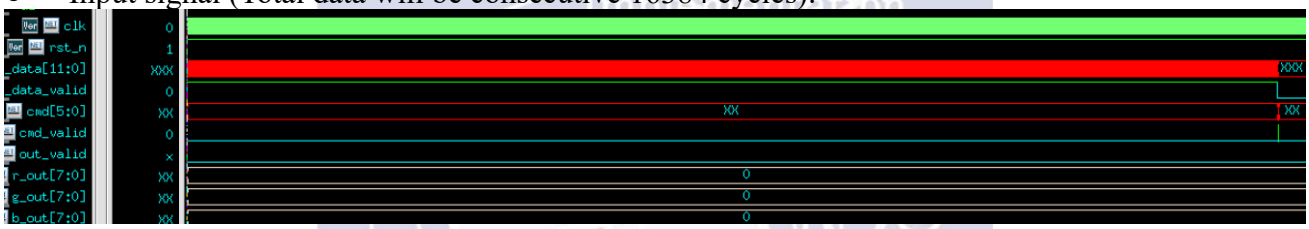
4. **Template folders and reference commands:**
   ➢ 01_RTL/ (RTL simulation) ./01_run_vcs_rtl
   ➢ 02_SYN/ (Synthesis) ./01_run_dc_shell
   - Check if there is any latch in your design in syn.log
   - Check the timing and the area of design in /Report/ISP.timing、ISP.area

   ➢ 03_GATE/ (Gate-level simulation) ./01_run_vcs_gate
   ➢ 04_MEM/ (Memory location)
   ➢ 05_APR/ (APR Folder) ./00_combine
   ./01_setenv
   ➢ 06_POST/ (Post-level simulation) ./01_run_vcs_post
   ➢ 09_SUBMIT/ (Submit your files) ./00_tar
   ./01_submit

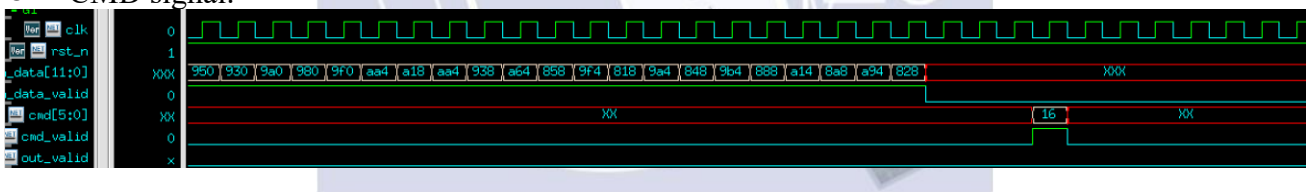
./02_check

Sample Waveform

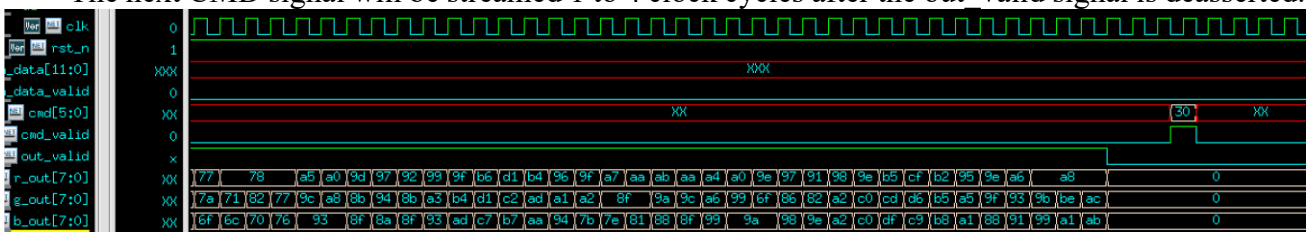
- Input signal (Total data will be consecutive 16384 cycles):



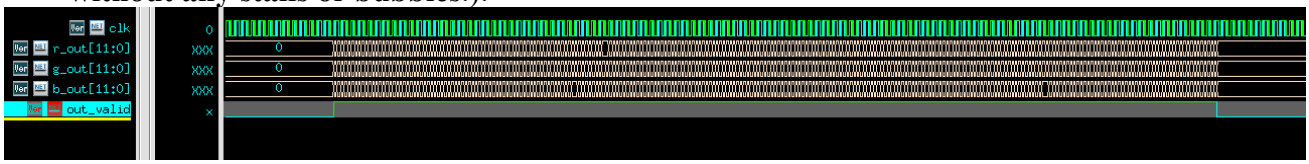
- CMD signal:



- The next CMD signal will be streamed 1 to 4 clock cycles after the out_valid signal is deasserted.



- Output signal(The output signals r_out, g_out, and b_out must operate like a **continuous stream**; once the first pixel is asserted, the pipeline must provide **256 consecutive cycles of valid data without any stalls or bubbles**.):



Reference

[1] C.-Y. Cho, W.-S. Wang, T.-M. Chen, and C.-N. Liu, "Real-Time Photo Sensor Dead Pixel Detection for Embedded Devices," in *Proceedings of the Industrial Technology Research Institute (ITRI)*, Hsinchu, Taiwan.